

DAY5

울산_4반 이름: 박인우

Chapter 1. 오늘 내용 정리 및 개인 이해

게시판이라는 기본 서비스에 댓글·좋아요라는 상호작용을 추가하고, REST API로 엔드포인트를 설계한 뒤, Spring Security + JWT로 접근을 보호하고, 완성된 애플리케이션을 컨테이너로 빌드·배포하면서 전체 서비스 구조를 이해

기존 JWT 인증 기반 Spring Boot 프로젝트에 댓글·좋아요 도메인을 계층형 구조로 확장하고, 인증된 사용자가 각 서비스에 접근하는 흐름을 연결한 뒤, 전체 애플리케이션을 Docker 이미지로 빌드하여 컨테이너 환경에서 실행

Spring Security의 Filter Chain을 통해 Controller 이전에 인증·인가를 처리하고, 세션 기반 인증에서 JWT 기반 Stateless 인증으로 확장한 뒤, MSA에서는 인증 서버를 분리하여 토큰 발급 → 검증 → 서비스 사용 구조로 확장할 수 있음을 학습

Spring Security/JWT로 입구를 보호한다 → 인증된 사용자가 Post·Comment·Like 같은 실제 비즈니스 기능을 사용한다 → 완성된 서버를 컨테이너로 실행한다

내이해 : 게시판을 만든다 → 댓글을 추가한다 → 사용자 간 상호작용이 생긴다 → 좋아요를 추가한다 → 사용자 반응이라는 새로운 가치가 생긴다 → REST API로 기능을 외부에 제공한다 → Spring Security + JWT로 누가 사용할 수 있는지 통제한다 → Docker Container로 동일한 환경에서 빌드·배포한다 → 서비스가 커지면 MSA로 독립적인 서비스를 구성할 수 있다

Chapter 2. 수업 내용

1. 엔드포인트 설계 원칙

“어떤 사용자가 어떤 기능에 접근할 수 있는가?”까지 고려해서 API의 입구를 설계하는 것

- `/login`, `/signup` 같은 인증 없이 접근 가능한 경로
- `/home`, `/users/**` 같은 인증이 필요한 경로

- 요청이 **Controller**에 바로 들어가는 것이 아니라, 먼저 **Security Filter Chain**을 통과하도록 설계
- 인증 결과에 따라 실제 서비스 접근 허용 여부를 결정

Client 요청 → Security Filter Chain → 인증/인가 검사 → Controller → Service → Repository / DB

2. Spring Security

인증(Authentication)과 인가(Authorization)에 특화된 Spring의 보안 프레임워크.

- 인증: 로그인한 사용자가 누구인지 확인
- 인가: 해당 사용자가 특정 **API**에 접근할 권한이 있는지 확인

Spring Security는 여러 개의 Filter를 연결한 Filter Chain을 이용해 Controller 이전에 이 작업을 수행한다.

3. JWT 필터 체인 구현 및 통합

JWT 방식을 사용하면 요청마다 토큰을 검사하는 필터를 Security에 추가할 수 있다.

요청 → JwtAuthenticationFilter → JWT 존재 확인 → JWT 검증 → 사용자 정보 추출 → SecurityContext에 인증정보 저장 → SecurityFilterChain 인가 판단 → Controller 접근

JWT 필터가 사용자를 인증하고, Spring Security가 그 인증 결과를 이용해 접근 가능 여부를 판단한다.

4. 쿠키 + 세션 방식

전통적인 로그인 방식.

로그인 성공 시 서버가 Session ID를 만들고, 브라우저에게 쿠키로 전달한다.

로그인 → 서버 → Session 생성 → Session ID 발급 → Cookie에 저장 → 다음 요청 Cookie(Session ID) → 서버 → Session 조회 → 사용자 확인

실제 로그인 정보가 서버 세션에 저장된다. 따라서 쿠키에는 Session ID만 있고, 인증 상태는 서버가 가지고 있다.

5. JWT 토큰 기반 인증

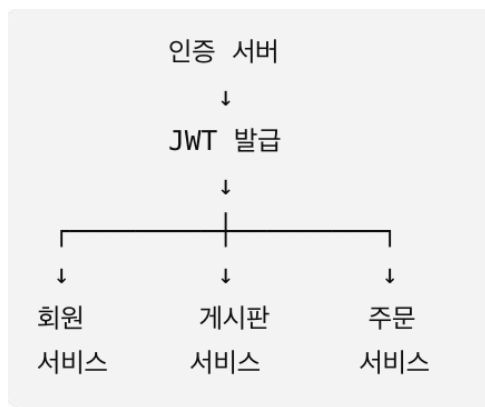
JWT에서는 서버의 **Session** 대신 토큰 자체에 인증에 필요한 정보를 담는다.

로그인 → ID/PW 검증 → JWT 발급 → Client가 JWT 보관 → 다음 요청 JWT 전달 → JWT Filter → 서명 / 만료시간 검증 → 사용자 식별 → API 접근

세션 방식	JWT 방식	
서버가 인증 상태 저장	토큰에 인증 정보 포함	
Session ID 사용	JWT 사용	
Stateful	Stateless에 적합	
서버에서 Session 조회	토큰 검증	
단일 서버에서 단순	분산 서버/MSA에 유리	

6. MSA와 인증 서버

MSA에서는 여러 서비스마다 로그인을 따로 구현하면 매우 비효율적이다.



인증 서버가 로그인과 토큰 발급을 담당하고, 각 서비스는 전달받은 **JWT**를 검증해서 사용자를 확인할 수 있다.

따라서 역할을 다음처럼 분리

① 발급

Authentication Server

로그인 성공 → JWT 발급

② 검증

JWT / Security Filter

토큰이 정상인지 확인

③ 사용

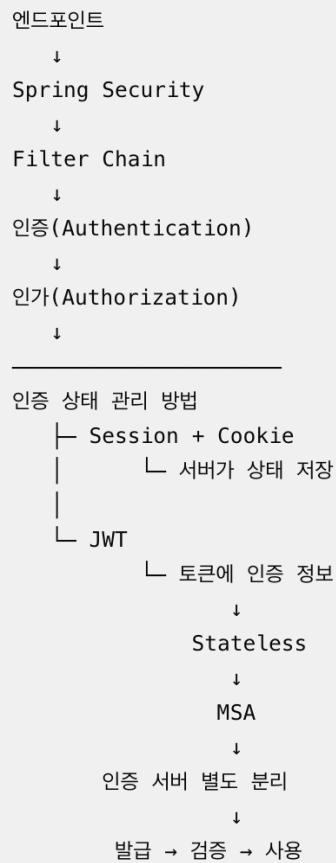
각 Service

검증된 사용자 정보를 기반으로 기능 수행

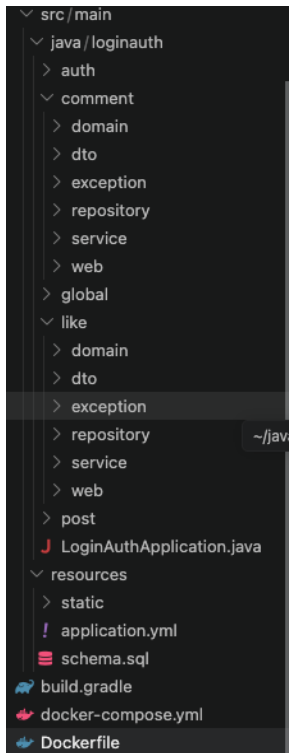
모놀리식 : 하나의 애플리케이션 안에 모든게 다 들어감, 하나의 JAR로 같이 빌드하고 같이 배포

Microservices : 서비스 나누고 배포도 따로, 각각의 JAR

7. 최종 흐름 정리



Chapter 3. 코드 구조 이해



기존 인증 구조 → 댓글(Comment) 추가 → 좋아요(Like) 추가 → 인증과 각 기능 연결 → Docker 이미지/컨테이너 실행, 기능(Comment / Like / Post) 단위로 먼저 나누고 그 안에 계층을 둠

댓글·좋아요 내부 요청 흐름

```

HTTP 요청
  ↓
Security Filter Chain
  ↓
JWT 인증
  ↓
web
Controller
  ↓
service
비즈니스 로직
  ↓
repository
DB 접근
  ↓
domain
데이터 객체
  
```

이 구조는 똑같다, 댓글 작성이나 좋아요도 마찬가지로,

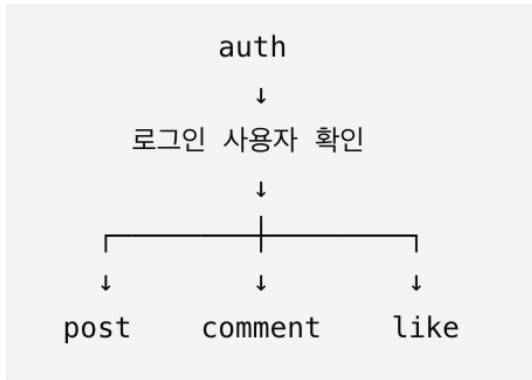


DTO / Domain / Repository / Service / Web 역할

계층	역할
web	HTTP 요청 받고 응답
dto	요청/응답 데이터 전달
service	댓글 작성, 좋아요 처리 등 실제 규칙
repository	DB 저장/조회
domain	Comment, Like 같은 핵심 객체
exception	해당 기능에서 발생하는 예외

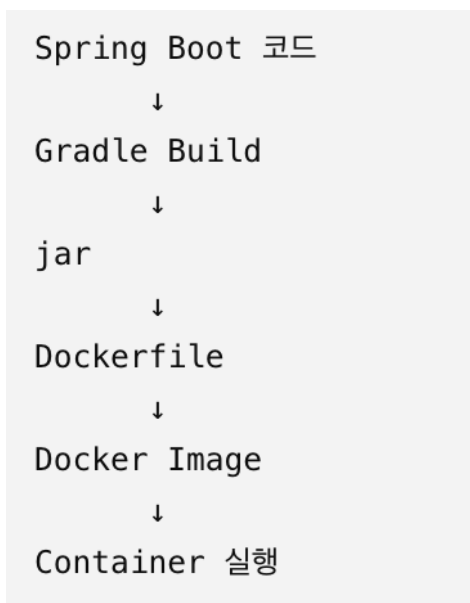
auth와 Comment/Like의 관계

comment나 like가 로그인 기능을 또 만드는 게 아님



인증은 **auth**에서 한 번 처리(JWT → 사용자 A 확인), 그 결과를 이용해서 게시물 댓글, 좋아요 동작

Docker 실행으로 실제 서버 배포에 가까운 실행 환경을 만들어봄



Chapter 4. 궁금했던 점 및 보충

Rest API

클라이언트가 서버의 데이터를 URL로 지정하고, HTTP 메서드로 무엇을 할지 표현하는 API 방식

Method	의미	예
GET	조회	게시글 조회
POST	생성	게시글/댓글 작성
PUT	전체 수정	게시글 전체 변경
PATCH	일부 수정	제목만 수정
DELETE	삭제	댓글 삭제

URL은 자원(Resource)을 나타내고, HTTP Method가 그 자원에 무엇을 할지를 나타낸다.

Container는 왜 사용하는가?

컨테이너는 쉽게 말하면:

애플리케이션과 실행에 필요한 환경을 이미지로 함께 묶어서 어디서든 비슷하게 실행시키는 단위

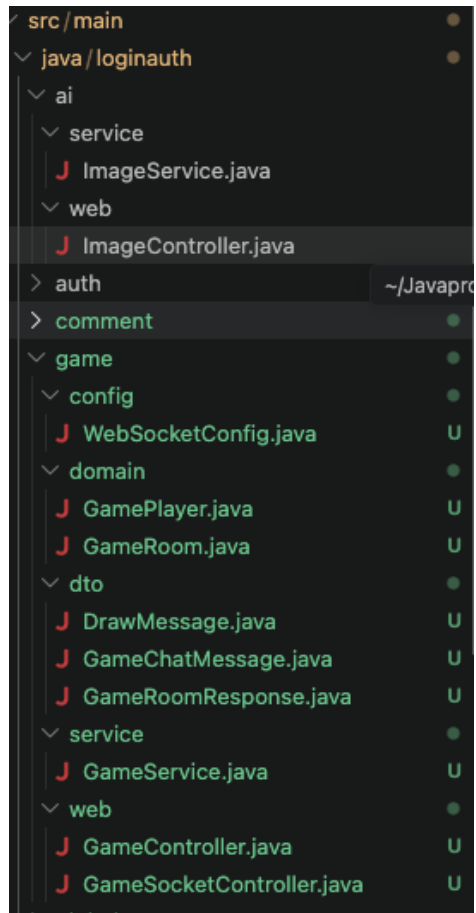
Kubernetes?

Docker → 컨테이너를 만든다, Kubernetes → 수많은 컨테이너를 운영·관리한다

특히 MSA가 되면, 컨테이너가 많아질 수 있음

컨테이너 실행, 컨테이너 재시작, 트래픽 분배, Scale Out, 배포 상태 관리 등을 담당해 준다.

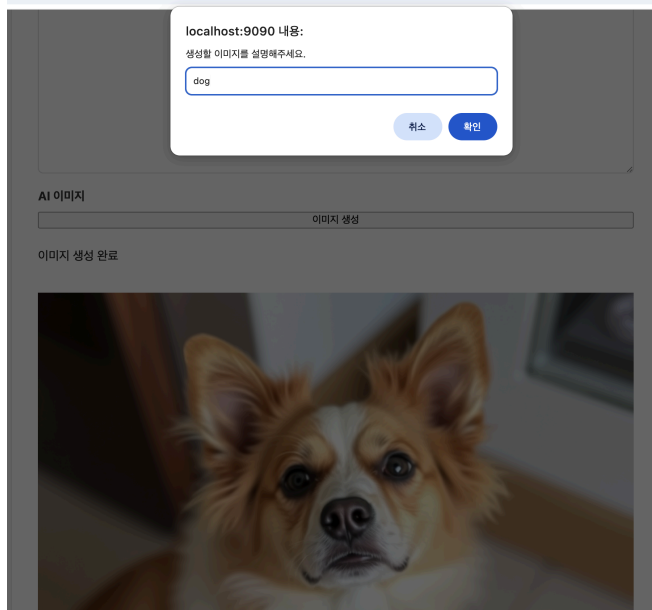
부록1. Ai이미지 생성, 캐치마인드 게임 만들어서 볼여보기



1. 게시판에 이미지 생성해서 올릴수 있게 하기

기존 Spring Boot 게시판의 `Controller → Service → Repository → DB` 계층 구조는 유지하면서, 별도의 `ImageController → ImageService → Cloudflare Workers AI(FLUX.1 Schnell)` 흐름을 추가해 **게시글 작성 중 AI 이미지를 생성하고, 생성된 이미지는 서버

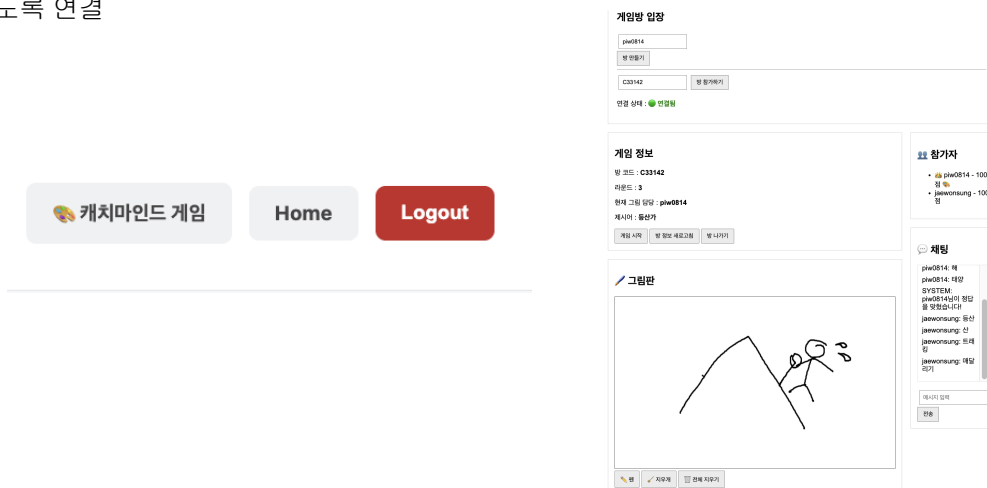
파일시스템에 저장하며, DB에는 이미지 경로만 저장하여 게시글 조회 시 함께 표시하도록 확장



2. 캐치마인드 만들기

기존 Spring Boot 게시판 프로젝트에 캐치마인드 게임 기능을 연결. 게임방과 플레이어 구조를 만들고, WebSocket을 이용해 그림과 채팅 메시지를 실시간으로 주고받도록 구성했다. 또한 Controller, Service, DTO, Domain, Config 역할을 구분해 구조를 정리했고, 기존 인증·게시판 기능과 함께 하나의 서비스 안에서 동작하도록 통합.

`ngrok`을 사용해 로컬 Spring Boot 서버를 외부에 공개하여 다른 사용자가 실제로 접속해 게임할 수 있도록 연결



부록2. 수업 과정 중 느낀점들

핵심은 “Java 개발자가 되는 수업”이라기보다, AI 기능을 실제 업무 서비스 안에 넣기 위해 Java/Spring 백엔드 구조를 이해하는 것

AI 모델 자체는 Python/FastAPI로 만들어도 상관없음. 업무 서비스 전체를 통제하는 중심이 Spring Boot가 됨

그래서 Java가 담당하는 건 이런 부분들

누가 이 AI를 사용할 수 있는가 → Security / Authentication

어떤 데이터를 AI에 보낼 것인가 → DTO / Service

업무 데이터를 어디서 가져오는가 → Repository / DB

언제 AI를 호출할 것인가 → Service

AI가 실패하면 어떻게 할 것인가 → Exception

AI 결과를 누구에게 어떤 형태로 보여줄 것인가 → Controller / REST API

기존 업무 기능과 AI 기능을 어떻게 연결할 것인가 → Architecture / DI

즉 AI 모델만 잘 만드는 것과 AI 서비스를 만드는 것은 다름

예를 들어 Python에서: `result = model.predict(data)`이 한 줄이 있다고 해도 실제 서비스에서는 그 앞뒤에 엄청 많은 게 붙는다

로그인된 사용자인가? → 이 데이터를 볼 권한이 있는가? → DB에서 어떤 데이터를 가져올까? → 전처리는? → AI 서버 호출 → Timeout 나면? → AI 결과 검증 → DB 저장 → 사용자에게 어떤 JSON으로 줄까?

바로 이 부분이 지금 배우는 Spring 백엔드

앞으로 코드를 볼 때는 이런 6가지 질문

이 객체의 역할은 무엇인가?

어디서 데이터를 받아오는가?

누구에게 일을 넘기는가?

무엇을 반환하는가?

인증·DB·예외처리는 어디에서 담당하는가?

여기에 AI 기능을 붙인다면 어느 지점에 들어갈까?